# 0.24. Les deux axes à la suite

deplace_x puis deplace_y, sans « = » : la lettre va à droite, puis descend depuis la colonne où elle est arrivée.

## 1. Ce qu'il faut comprendre

**On garde la forme courte du 0.23**, et on utilise **les deux axes** : d'abord `deplace_x`, puis `deplace_y`.

**Ce qui est nouveau ici :** deux variables, `x` **et** `y`, et chaque fonction reçoit **les deux**. `deplace_x(x, y, …)` avance sur la ligne `y` ; `deplace_y(x, y, …)` descend dans la colonne `x`.

**C'est pour cela qu'il faut garder `x` à jour :** `deplace_y` part de la colonne `x`, celle où `deplace_x` a laissé la lettre (5). Si `x` était resté à 0, la descente partirait de la colonne 0, et la lettre sauterait d'un coup.

**Les nombres**, en bas de l'écran (ligne 17) : `x` en colonne 0, `y` en colonne 4.

**Les fonctions de cette leçon — et le tuto de chacune**

- `#include <deplace_x>` → 0.17.3. La fonction deplace_x() — avancer sur une ligne
- `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
- `#include <nombre>` → 0.8.3. La fonction nombre() — écrire un nombre
- `#include <deplace_y>` → 0.20.3. La fonction deplace_y() — avancer sur une colonne

### 📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

### 🔍 Le programme, ligne par ligne — chaque ligne dite en français

| | |
|---|---|
| `#include <deplace_x>  // fait avancer une tuile sur sa ligne` | Demande à la console « deplace_x » (ce que c'est : fait avancer une tuile sur sa ligne). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « fait avancer une tuile sur sa ligne ».) |
| `#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A` | Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».) |

| | |
|---|---|
| `#include <nombre>  // écrit un nombre en chiffres` | Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un nombre en chiffres ».) |
| `#include <deplace_y>  // fait avancer une tuile sur sa colonne` | Demande à la console « deplace_y » (ce que c'est : fait avancer une tuile sur sa colonne). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « fait avancer une tuile sur sa colonne ».) |
| `uint8_t pas = 5;  // 5 cases, sur chaque axe` | Une variable : une boîte appelée pas, qui retient un nombre de 0 à 255, et qui commence à 5. (Le commentaire le dit : « 5 cases, sur chaque axe ».) |
| `uint8_t x = 0;  // la COLONNE de la lettre (axe X)` | Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la COLONNE de la lettre (axe X) ».) |
| `uint8_t y = 0;  // la LIGNE de la lettre (axe Y)` | Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la LIGNE de la lettre (axe Y) ».) |
| `int main() {` | Le début du programme : la console commence ici, par la première ligne qui suit. |
| `// 1. À droite : deplace_x reçoit x ET y — elle avance sur la ligne y (0).` | Un commentaire : une phrase pour celui qui lit (« 1. À droite : deplace_x reçoit x ET y — elle avance sur la ligne y (0). »). La console l'ignore. |
| `deplace_x(x, y, ALPHABET[0], pas);  // x vaut 5` | Appel de la fonction deplace_x de la console, avec les arguments x, y, ALPHABET[0], pas. Ce qu'elle fait : pose la tuile, puis la fait bouger d'une case tous les quarts de seconde, « pas » fois : 5 avance vers la droite, -5 recule vers la gauche. S'arrête au bord de l'écran. Rend la colonne d'arrivée : x = deplace_x(x, 0, ALPHABET[0], 5). Seule sur sa ligne avec une variable, deplace_x(x, 0, ALPHABET[0], 5); range elle-même la colonne dans x. (Le commentaire le dit : « x vaut 5 ».) |
| `nombre(0, 17, x);  // en bas à gauche : 005` | Appel de la fonction nombre de la console, avec les arguments 0, 17, x. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « en bas à gauche : 005 ».) |
| `// 2. En bas : deplace_y part de la colonne x — 5, là où la lettre est.` | Un commentaire : une phrase pour celui qui lit (« 2. En bas : deplace_y part de la colonne x — 5, là où la lettre est. »). La console l'ignore. |
| `//   deplace_y(x, y, ALPHABET[0], pas);` | Un commentaire : une phrase pour celui qui lit (« deplace_y(x, y, ALPHABET[0], pas); »). La console l'ignore. |
| `//  \|  \|` | Un commentaire : une phrase pour celui qui lit (« \|  \| »). La console l'ignore. |
| `//  \|  +-- y : la ligne de départ (0) ; la console y range l'arrivée (5)` | Un commentaire : une phrase pour celui qui lit (« \| +-- y : la ligne de départ (0) ; la console y range l'arrivée (5) »). La console l'ignore. |
| `//  +----- x : la colonne, 5 grâce à la ligne d'avant` | Un commentaire : une phrase pour celui qui lit (« +----- x : la colonne, 5 grâce à la ligne d'avant »). La console l'ignore. |
| `deplace_y(x, y, ALPHABET[0], pas);  // y vaut 5` | Appel de la fonction deplace_y de la console, avec les arguments x, y, ALPHABET[0], pas. Ce qu'elle fait : comme deplace_x(), mais sur l'axe Y : 5 descend de 5 lignes, -5 monte de 5 lignes. S'arrête au bord de l'écran (ligne 0 ou 17). Rend la ligne d'arrivée : y = deplace_y(0, y, ALPHABET[0], 5). Seule sur sa ligne avec une |

| | |
|---|---|
| | variable, deplace_y(0, y, ALPHABET[0], 5); range elle-même la ligne dans y. (Le commentaire le dit : « y vaut 5 ».) |
| `nombre(4, 17, y);  // à côté : 005` | Appel de la fonction nombre de la console, avec les arguments 4, 17, y. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « à côté : 005 ».) |
| `while (true) {` | La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). |
| `image();  // la lettre est en (5, 5)` | Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « la lettre est en (5, 5) ».) |
| `}` | L'accolade fermante : la fin du bloc ouvert plus haut par « { ». |
| `}` | L'accolade fermante : la fin du bloc ouvert plus haut par « { ». |

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1306 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        66 o  ← la console, à l'allumage  — la mise en route de la console
  deplace_x()     158 o  ← main()  — une fonction fournie par le langage
  deplace_y()     158 o  ← main()  — une fonction fournie par le langage
  main()           69 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases, EcrireNombre  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  EcrireNombre     33 o  ← nombre()  — écrit un nombre en chiffres
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert,
Notes
Fonctions du langage appelées :
  poser()     × 4    52 o sur place, dans deplace_x(), deplace_y()  + AdresseCase (10 o)
  effacer()   × 2    30 o sur place, dans deplace_x(), deplace_y()  + AdresseCase (10 o), EffacerCases (12 o)
  nombre()    × 2    20 o sur place, dans main()  + EcrireNombre (33 o)
  image()     × 3     9 o sur place, dans deplace_x(), deplace_y(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  deplace_x() × 1    19 o sur place, dans main()
  deplace_y() × 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x>   // fait avancer une tuile sur sa ligne
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <nombre>      // écrit un nombre en chiffres
#include <deplace_y>   // fait avancer une tuile sur sa colonne
```

#### 2. UNE VARIABLE — PAS

```
uint8_t pas = 5;      // 5 cases, sur chaque axe
```

#### 3. UNE VARIABLE — X

```
uint8_t x = 0;        // la COLONNE de la lettre (axe X)
```

#### 4. UNE VARIABLE — Y

```
uint8_t y = 0;        // la LIGNE de la lettre (axe Y)
```

#### 5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  // 1. À droite : deplace_x reçoit x ET y — elle avance sur la ligne y (0).
  deplace_x(x, y, ALPHABET[0], pas);    // x vaut 5
  nombre(0, 17, x);                     // en bas à gauche : 005

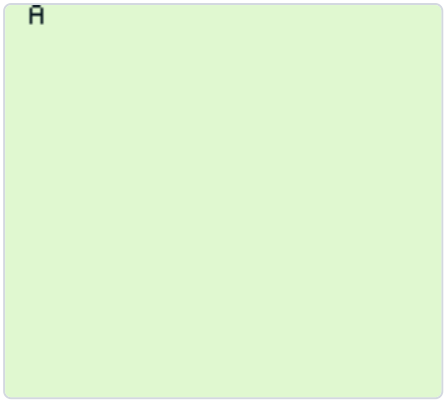
  // 2. En bas : deplace_y part de la colonne x — 5, là où la lettre est.
  //
  //   deplace_y(x, y, ALPHABET[0], pas);
  //             |  |
  //             |  +-- y : la ligne de départ (0) ; la console y range l'arrivée (5)
  //             +----- x : la colonne, 5 grâce à la ligne d'avant
  deplace_y(x, y, ALPHABET[0], pas);    // y vaut 5
  nombre(4, 17, y);                     // à côté : 005

  while (true) {
    image();          // la lettre est en (5, 5)
  }
}
```

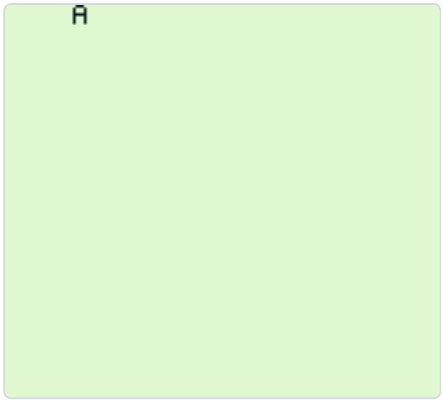
## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

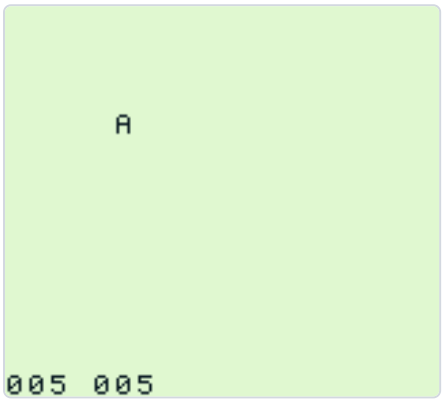
la toute première image — la console vient de démarrer — image 1




un tiers de seconde plus tard : le premier dessin est posé — image 20




au bout d’une seconde — image 60




au bout de trois secondes — image 180

## 4. Ce qu’on doit voir

> Un A qui va de 5 cases à droite, puis descend de 5 lignes. En bas, 005 et 005.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ à droite d’abord, jusqu’en (5, 0)
- ✓ puis en bas depuis la colonne 5 : (5, 3)
- ✓ x et y valent 5
- ✓ 005 et 005 affichés (« 005 005 »)

---

Ensuite : **0.24.1. Les deux axes à la suite — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++